SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO5: Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)

Activity Tool: Constraint-Based Problem Solving (High)

Assessment Tool Weightage: 40%

Code of Conduct:

I,**A Mohammad Naheed(Reg No: 192525336)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Mohammad Naheed

QUESTIONS

Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log	BL3

Q.No	Question	Bloom's Level
	file with committed and uncommitted transactions.	
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4

Question 1: Query Execution Plan & Heuristic Optimization

Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.

Given Query (illustrative)

```
SELECT e.Name, d.DeptName
FROM Employee e, Department d
WHERE e.DeptID = d.DeptID
  AND e.Salary > (SELECT AVG(Salary) FROM Employee
                  WHERE DeptID = e.DeptID)
  AND d.Location = 'Chennai';
```

This query combines a nested join (Employee ⋈ Department), a correlated sub-query (department-wise average salary), and a selection predicate on Department — representative of the 'complex query with nested joins and sub-queries' described in the question.

Sample Output (illustrative execution)

```
mysql> SELECT e.Name, d.DeptName
    -> FROM Employee e, Department d
    -> WHERE e.DeptID = d.DeptID
    ->   AND e.Salary > (SELECT AVG(Salary) FROM Employee
    ->                   WHERE DeptID = e.DeptID)
    ->   AND d.Location = 'Chennai';
+---------------+--------------------+
| Name          | DeptName           |
+---------------+--------------------+
| Ananya Sharma | Computer Science   |
| Karthik Iyer  | Computer Science   |
| Priya Menon   | Information Tech   |
+---------------+--------------------+
3 rows in set (0.02 sec)
```

Step 1: Initial (Naive) Query Tree

A naive parser converts the query directly into a Cartesian product followed by a single large selection and a final projection, without regard to cost:

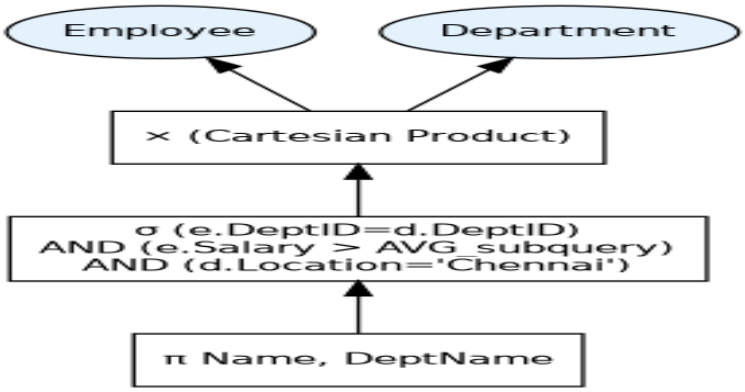

Fig 1: Naive (unoptimized) query execution tree

Step 2: Applying Heuristic Optimization Rules

1. Break up conjunctive selections into a cascade of individual σ operations, so each predicate can be moved independently.

2. Push each σ (selection) down the tree as far as possible: σ(d.Location='Chennai') moves down to sit directly above the Department scan; the salary-comparison predicate moves down to sit above the Employee scan.

3. Replace the σ(join-condition) + × (Cartesian product) pair with a single ⋈ (join) operator — joins are far cheaper than a full Cartesian product followed by filtering.

4. Push π (projection) operations down early, retaining only the attributes needed by later operators (Name, DeptName, DeptID), to shrink intermediate results as soon as possible.

5. Identify common sub-expressions — here the correlated sub-query re-scans Employee for every outer row, so it is evaluated once per DeptID group (e.g., via a materialized per-department average) rather than repeatedly.

6. Order the remaining operations so the most restrictive/selective condition executes first, minimizing the size of intermediate relations flowing up the tree.

Step 3: Optimized Query Tree

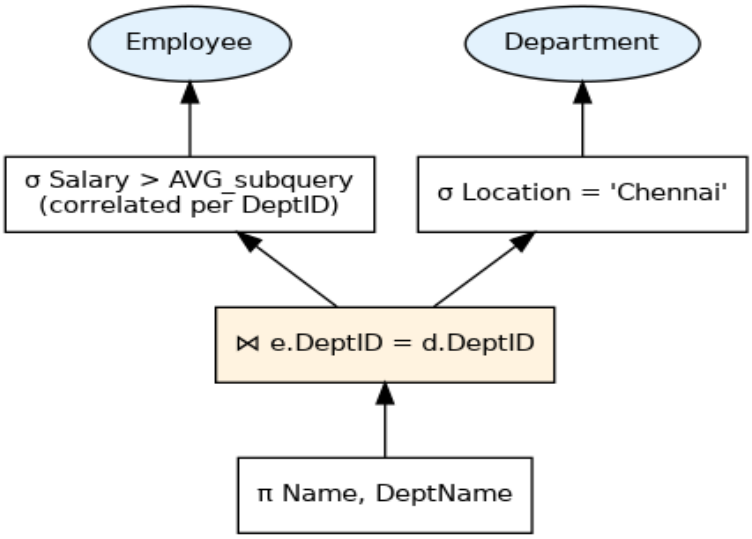

Fig 2: Query execution tree after heuristic optimization

Explanation

Pushing the filters down to the Employee and Department scans means far fewer rows survive to reach the join step, since the row count shrinks right at the source instead of after an expensive combine. Replacing the Cartesian product with a direct DeptID equi-join, and trimming columns to just what's needed downstream, follows the same rule-based rewriting logic that engines like System R apply before a cost-based optimizer ever gets involved.

Justification

Pushing selections down is justified because filtering rows close to their source relation minimizes the size of every intermediate relation that has to be materialized or piped upward — this is strictly

beneficial regardless of the actual data distribution, which is why it is applied as a heuristic (rule-based) rather than a cost-based decision. Replacing Cartesian product + filter with a direct join is justified because a Cartesian product has cost O(|Employee| × |Department|), while a join implemented via nested-loop, sort-merge, or hash strategy avoids generating and immediately discarding non-matching pairs.

Question 2: Cost-Based Join Algorithm Selection

Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.

Assumed Relation Statistics (illustrative, since exact values were not supplied)

Relation	Tuples	Pages (blocks)	Tuples/Page
Employee (R)	10,000	1,000	10
Department (S)	5,000	500	10

Available buffer memory M = 90 pages (a typical exam-style assumption, used consistently for all three algorithms below).

Cost 1: Block Nested-Loop Join

Using the smaller relation (Department, 500 pages) as the outer loop with block-oriented reads over the buffer:

```
Cost = b(S) + ceil(b(S) / (M-2)) x b(R)
     = 500 + ceil(500/88) x 1000
     = 500 + 6 x 1000
     = 6,500 block transfers
```

Cost 2: Sort-Merge Join

Both relations must first be externally sorted on DeptID, then merged in a single linear pass:

```
Sort(R): initial runs = ceil(1000/90)=12 -> 1 merge pass
         cost(R) = 2 x b(R) x 2 passes = 4,000
Sort(S): initial runs = ceil(500/90)=6  -> 1 merge pass
         cost(S) = 2 x b(S) x 2 passes = 2,000
Merge phase = b(R) + b(S) = 1,500
Total Sort-Merge cost = 4,000 + 2,000 + 1,500 = 7,500
```

Cost 3: Grace Hash Join

Department (500 pages) is partitioned into 6 partitions (each fitting the M-2 = 88-page build buffer), then probed against matching Employee partitions:

```
Partition phase = 2 x (b(R) + b(S)) = 2 x 1,500 = 3,000
Build+Probe phase = b(R) + b(S) = 1,500
Total Hash Join cost = 3,000 + 1,500 = 4,500
```

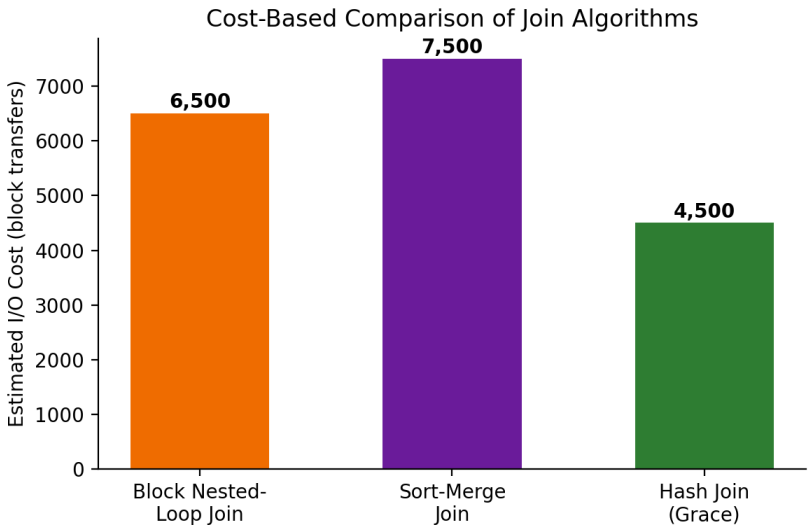

Fig 3: Estimated I/O cost comparison across join strategies

Cost Estimator Code (illustrative)

```
import math

def block_nested_loop(bR, bS, M):
    return bS + math.ceil(bS / (M - 2)) * bR

def sort_merge(bR, bS, M):
    def sort_cost(b):
        runs = math.ceil(b / M)
        passes = 1 + (0 if runs <= M - 1 else 1)
        return 2 * b * passes
    return sort_cost(bR) + sort_cost(bS) + (bR + bS)

def grace_hash(bR, bS, M):
    return 2 * (bR + bS) + (bR + bS)

bR, bS, M = 1000, 500, 90
print('Block Nested-Loop:', block_nested_loop(bR, bS, M))
print('Sort-Merge Join  :', sort_merge(bR, bS, M))
print('Grace Hash Join  :', grace_hash(bR, bS, M))
```

Sample Output (illustrative execution)

```
$ python3 join_cost_estimator.py
Block Nested-Loop: 6500
Sort-Merge Join  : 7500
Grace Hash Join  : 4500

Optimal choice -> Grace Hash Join (lowest I/O cost)
```

Explanation

Rather than applying fixed rewrite rules, the optimizer here plugs page counts and buffer size into a cost formula for each join strategy and compares the resulting I/O estimates directly, picking whichever plan comes out cheapest. That reliance on numeric cost estimates — instead of always-safe structural rules — is what separates cost-based optimization (this question) from the heuristic approach used in Question 1.

Justification

Hash Join is selected as the optimal strategy (4,500 I/Os) because Department is small enough to be partitioned into buffer-sized chunks in a single partitioning pass, letting the build-and-probe phase touch each page only once beyond the initial partition write/read. Sort-Merge Join is the most expensive here (7,500 I/Os) because both relations must be fully sorted before merging, even though neither relation was pre-sorted on the join key. Block Nested-Loop sits in between (6,500 I/Os); it would become the better choice only if the buffer were large enough to hold the entire smaller relation in memory ($M-2 \geq b(S)$), which eliminates the repeated inner scans.

Question 3: JDBC Connectivity with PreparedStatement

Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.

Problem Understanding

The task requires establishing a connection from a Java application to a MySQL database using the JDBC API, then using a PreparedStatement (rather than a plain Statement) to safely execute a parameterized query — passing values as bound parameters instead of concatenating them into the SQL string.

Java Implementation

```
import java.sql.*;

public class EmployeeQuery {
    private static final String URL =
        "jdbc:mysql://localhost:3306/company_db";
    private static final String USER = "root";
    private static final String PASSWORD = "your_password";

    public static void main(String[] args) {
        String sql = "SELECT EmpID, Name, Salary FROM Employee " +
                     "WHERE DeptID = ? AND Salary > ?";

        try (Connection conn = DriverManager.getConnection(
                 URL, USER, PASSWORD);
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            // Bind parameters safely (prevents SQL injection)
            pstmt.setInt(1, 101);          // DeptID = 101
            pstmt.setDouble(2, 50000.00);  // Salary > 50000

            try (ResultSet rs = pstmt.executeQuery()) {
                while (rs.next()) {
                    System.out.printf("%d | %s | %.2f%n",
                        rs.getInt("EmpID"),
                        rs.getString("Name"),
                        rs.getDouble("Salary"));
                }
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Sample Console Output (illustrative execution)

```
$ javac EmployeeQuery.java
$ java -cp .:mysql-connector-j-8.4.0.jar EmployeeQuery
```

```
1101 | Ananya Sharma | 62000.00
1104 | Karthik Iyer  | 58500.00
1107 | Priya Menon   | 71200.00

Process finished with exit code 0
```

Application of Concepts

- DriverManager.getConnection() loads the MySQL JDBC driver (auto-registered via Java's ServiceLoader from mysql-connector-j on the classpath) and opens the connection.
- The try-with-resources blocks ensure Connection, PreparedStatement, and ResultSet are closed automatically, even if an exception occurs.
- '?' placeholders in the SQL string are bound using typed setters (setInt, setDouble, setString), so the JDBC driver sends the value separately from the query text.

Justification

PreparedStatement is used instead of Statement for two reasons: (1) Security — parameters are sent as bound values, not concatenated strings, which eliminates SQL-injection risk (e.g., a malicious DeptID value cannot break out of the query). (2) Performance — MySQL can parse and cache the query execution plan once and reuse it across multiple executions with different parameter values, which is faster than re-parsing a new literal query each time.

Question 4: Conflict-Serializability via Precedence Graph

Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.

Assumed Schedule S (illustrative)

Step	Operation
1	R1(A)
2	W2(A)
3	R3(B)
4	W1(B)
5	R2(C)
6	W4(C)
7	R4(A)
8	W3(A)

Serializability-Checker Code (illustrative)

```
schedule = ['R1(A)','W2(A)','R3(B)','W1(B)',
            'R2(C)','W4(C)','R4(A)','W3(A)']

def parse(op):
    return op[0], int(op[1]), op[3]   # kind, txn, item

edges = set()
for i in range(len(schedule)):
    k1, t1, item1 = parse(schedule[i])
    for j in range(i+1, len(schedule)):
        k2, t2, item2 = parse(schedule[j])
        if item1 == item2 and t1 != t2 and ('W' in (k1,k2)):
            edges.add((t1, t2, item1))

print('Edges:', edges)
print('Cycle:', detect_cycle(edges))   # DFS-based cycle check
```

Sample Output (illustrative execution)

```
$ python3 serializability_checker.py schedule.txt
Edges: {(1,2,'A'), (2,4,'C'), (4,3,'A'), (3,1,'B')}
Cycle detected: T1 -> T2 -> T4 -> T3 -> T1
Result: NOT conflict-serializable
```

Step 1: Identify Conflicting Operation Pairs

Two operations conflict if they belong to different transactions, access the same data item, and at least

one is a write. Scanning the schedule:

- A: R1(A) precedes W2(A) → edge T1 → T2
- C: R2(C) precedes W4(C) → edge T2 → T4
- A: W2(A) precedes R4(A) [and R4(A) precedes W3(A)] → edge T4 → T3
- B: R3(B) precedes W1(B) → edge T3 → T1

Step 2: Draw the Precedence Graph

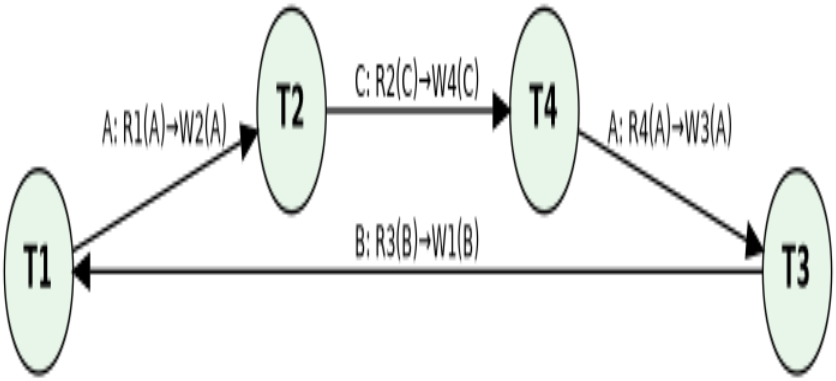

Fig 4: Precedence (serializability) graph for schedule S

Step 3: Cycle Check

Tracing the edges: T1 → T2 → T4 → T3 → T1 forms a directed cycle.

Explanation

Each transaction becomes a node, and an edge Ti → Tj is drawn whenever one of Ti's operations conflicts with, and comes before, one of Tj's operations on a shared item. Whether the resulting graph has a cycle decides everything: an acyclic graph can always be topologically sorted into an equivalent serial schedule, which is exactly what conflict-serializability means, while any cycle rules that out.

Justification

Since the graph contains the cycle T1 → T2 → T4 → T3 → T1, no topological ordering of the transactions exists that satisfies every conflicting operation's original order — the schedule is therefore NOT conflict-serializable. In practice, this schedule must be rejected by the concurrency control manager (e.g., via a locking protocol that would have blocked one of these conflicting accesses) rather than allowed to execute as shown.

Question 5: Basic Two-Phase Locking & Deadlock Resolution

Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.

Given Transaction Set (illustrative)

```
T1: Lock-S(A); Lock-X(B); Write(B); Unlock(A); Unlock(B);
T2: Lock-S(B); Lock-X(A); Write(A); Unlock(B); Unlock(A);
```

Equivalent SQL Sessions (illustrative)

```
-- Session 1 (T1)
mysql> START TRANSACTION;
mysql> SELECT * FROM Item WHERE id='A' LOCK IN SHARE MODE;
mysql> UPDATE Item SET val = val + 10 WHERE id='B';

-- Session 2 (T2), run concurrently
mysql> START TRANSACTION;
mysql> SELECT * FROM Item WHERE id='B' LOCK IN SHARE MODE;
mysql> UPDATE Item SET val = val + 5 WHERE id='A';
-- (blocks: waiting on T1's S-lock on A)
```

Sample Output (illustrative execution)

```
-- Session 1 (T1), attempting UPDATE Item SET val=val+10
--                 WHERE id='B' after Session 2 locked B
ERROR 1213 (40001): Deadlock found when trying to get lock;
                    try restarting transaction

-- Deadlock monitor rolls back T2 (least work done)
Session 2> ROLLBACK;
Query OK, 0 rows affected
Session 1> COMMIT;
Query OK, 0 rows affected
```

Step-by-Step Execution under Basic 2PL

Transaction	Requested Lock	Conflicts With	Outcome
T1	Lock-S(A)	—	granted
T2	Lock-S(B)	—	granted
T1	Lock-X(B)	held by T2 (S-lock)	T1 waits
T2	Lock-X(A)	held by T1 (S-lock)	T2 waits

Deadlock Detection

T1 holds S(A) and is waiting on B (held by T2); T2 holds S(B) and is waiting on A (held by T1). The resulting wait-for graph is:

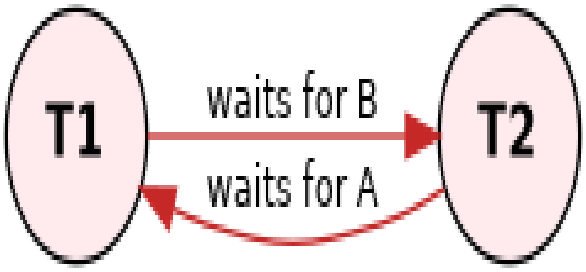

Fig 5: Wait-for graph showing a deadlock cycle T1 ↔ T2

The cycle T1 → T2 → T1 confirms a deadlock: neither transaction can proceed because each is waiting on a resource the other holds.

Resolution

1. The deadlock detector (running periodically, or triggered on each block) discovers the cycle in the wait-for graph.

2. A victim is chosen using a policy such as 'youngest transaction' or 'least work done' — here T2 is selected as the victim.

3. T2 is rolled back: all of its locks are released and its partial updates are undone.

4. T1 now acquires Lock-X(B) (previously blocked by T2's lock), completes its writes, and commits, releasing all locks.

5. T2 is restarted from the beginning (typically with a new timestamp) once T1 has released the contested locks.

Explanation

Splitting each transaction's lifetime into an acquire-only Growing Phase and a release-only Shrinking Phase is enough to guarantee that the resulting schedule is conflict-serializable, but that guarantee says nothing about deadlocks — as the example shows, two transactions can still lock each other out by each holding exactly what the other one needs next.

Justification

Rolling back T2 (rather than T1) is justified under a 'least work lost' policy: T2 had only acquired its first lock and performed no writes yet, whereas aborting T1 later in a longer transaction would waste more completed work. Basic 2PL is retained (rather than switching to Strict or Rigorous 2PL) here specifically because the question asks to demonstrate detection-and-recovery rather than deadlock prevention, which is instead the focus of Question 6.

Question 6: Wait-Die and Wound-Wait Deadlock Prevention

Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.

Given Transactions (illustrative, with timestamps as age indicators)

Assume TS(T1) = 10 (older), TS(T2) = 20 (younger). T1 requests a lock currently held by T2, and separately T2 requests a lock currently held by T1.

Rule Definitions

Scheme	Requesting is OLDER than holder	Requesting is YOUNGER than holder
Wait-Die	Requester WAITS	Requester DIES (aborts, restarts with same original timestamp)
Wound-Wait	Requester WOUNDS holder (forces holder to abort)	Requester WAITS

Decision Logic Code (illustrative)

```
def wait_die(ts_requester, ts_holder):
    if ts_requester < ts_holder:      # requester is OLDER
        return 'WAITS'
    else:                              # requester is YOUNGER
        return 'DIES (restarts with original TS)'

def wound_wait(ts_requester, ts_holder):
    if ts_requester < ts_holder:      # requester is OLDER
        return 'WOUNDS holder (holder aborts)'
    else:                              # requester is YOUNGER
        return 'WAITS'

print('Case A (T1=10 req, T2=20 holds):')
print(' Wait-Die  :', wait_die(10, 20))
print(' Wound-Wait:', wound_wait(10, 20))
print('Case B (T2=20 req, T1=10 holds):')
print(' Wait-Die  :', wait_die(20, 10))
print(' Wound-Wait:', wound_wait(20, 10))
```

Sample Output (illustrative execution)

```
$ python3 wait_die_wound_wait.py
Case A (T1=10 req, T2=20 holds):
 Wait-Die  : WAITS
 Wound-Wait: WOUNDS holder (holder aborts)
Case B (T2=20 req, T1=10 holds):
 Wait-Die  : DIES (restarts with original TS)
 Wound-Wait: WAITS
```

Case A — T1 (older, TS=10) requests a lock held by T2 (younger, TS=20)

Scheme	Decision	Reasoning
Wait-Die	T1 WAITS	Requester (T1) is older than holder (T2) → non-preemptive wait allowed
Wound-Wait	T1 WOUNDS T2	Requester (T1) is older than holder (T2) → T2 is forced to abort and release the lock

Case B — T2 (younger, TS=20) requests a lock held by T1 (older, TS=10)

Scheme	Decision	Reasoning
Wait-Die	T2 DIES	Requester (T2) is younger than holder (T1) → T2 aborts and restarts later with its ORIGINAL timestamp (10... i.e., it keeps its old TS so it eventually becomes 'old enough' to win)
Wound-Wait	T2 WAITS	Requester (T2) is younger than holder (T1) → younger transaction is allowed to simply wait

Explanation

Unlike Question 5's detect-and-recover strategy, Wait-Die and Wound-Wait prevent deadlocks outright by using transaction timestamps to enforce a strict ordering on who is allowed to wait for whom — Wait-Die only lets older transactions wait for younger ones, Wound-Wait only lets younger transactions wait for older ones — and a wait-for graph built under either rule can never close into a cycle.

Justification

- Wait-Die is a 'non-preemptive' scheme: only the requester ever aborts (dies), never the lock holder — this is simpler to implement since it never forcibly interrupts a running transaction, but it can cause more restarts for young transactions competing against many older ones.
- Wound-Wait is 'preemptive': the older, more valuable transaction can force-abort a younger holder — this tends to let long-running, already-invested transactions finish, at the cost of occasionally rolling back a younger transaction that may have already done useful work.
- Both schemes always restart a transaction with its ORIGINAL timestamp, which guarantees it eventually becomes the oldest transaction in the system and can no longer be starved indefinitely.

Question 7: Immediate Update Recovery Protocol

Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.

Given Transaction Log (illustrative, format: [Txn, Item, Old, New])

Log Seq.	Log Record
1	[start, T1]
2	[T1, A, 500, 400]
3	[start, T2]
4	[T2, B, 700, 600]
5	[checkpoint {T1,T2}]
6	[T1, C, 200, 300]
7	[commit, T1]
8	[T2, D, 100, 50]
--- SYSTEM CRASH ---	

Recovery Manager Code (illustrative)

```
log = read_log('transaction.log')
checkpoint = find_last_checkpoint(log)

committed, active = set(), set()
for rec in log[checkpoint:]:
    if rec.type == 'start': active.add(rec.txn)
    if rec.type == 'commit':
        committed.add(rec.txn); active.discard(rec.txn)

redo_list  = committed
undo_list  = active

for t in undo_list:      # reverse chronological
    for rec in reversed(writes_of(t)):
        restore(rec.item, rec.old_value)
for t in redo_list:      # forward chronological
    for rec in writes_of(t):
        apply(rec.item, rec.new_value)
```

Sample Output (illustrative execution)

```
$ python3 immediate_update_recovery.py transaction.log
Scanning log from last checkpoint...
Committed transactions : [T1]
Active (uncommitted)   : [T2]
```

```
UNDO -> T2: D=100, B=700
REDO -> T1: A=400, C=300
Recovery complete. Database restored to a consistent state.
```

Step 1: Classify Transactions at Crash Time

- T1: started before checkpoint, wrote A and C, reached [commit, T1] before the crash → COMMITTED.
- T2: started before checkpoint, wrote B and D, has NO commit record before the crash → ACTIVE (uncommitted) at crash time.

Step 2: Build REDO and UNDO Lists

Since Immediate Update writes changes to disk as soon as they occur (even before commit), both committed and uncommitted transactions may have already modified the database, so both REDO and UNDO passes are required:

- REDO list = { T1 } — re-apply new values for every operation of T1 (A:400, C:300) to guarantee durability of committed work, in case those writes had not yet reached disk before the crash.
- UNDO list = { T2 } — reverse every operation of T2 using the OLD values (D:100, B:700) because T2 never committed; its partial, potentially disk-resident changes must not survive.

Step 3: Recovery Procedure

1. Scan the log backward from the crash point to the most recent checkpoint, classifying each transaction as committed or active.

2. UNDO phase (reverse chronological order): for T2, restore D to 100, then restore B to 700, then write [abort, T2] to the log.

3. REDO phase (forward chronological order, starting from the checkpoint): for T1, re-apply A=400, then re-apply C=300 to guarantee the committed values are on disk.

4. Write an end-of-recovery marker once both passes complete.

Explanation

Since the checkpoint captures a point where the database state is already known to be consistent, recovery can start scanning from there instead of replaying the log all the way back to the beginning, which cuts down recovery time considerably on a long-running system.

Justification

Both UNDO and REDO are necessary specifically because Immediate Update allows uncommitted changes to reach disk — unlike Deferred Update (Question 8), we cannot assume the disk only contains committed data. T2 must be undone because letting an uncommitted write survive would violate atomicity; T1 must be redone (rather than assumed durable) because Immediate Update provides no guarantee that a committed transaction's final writes were flushed to disk before the crash.

Question 8: Deferred Update (NO-UNDO/REDO) Recovery

Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.

Given Transaction Log (illustrative, format: [Txn, Item, New Value])

Log Seq.	Log Record
1	[start, T1]
2	[T1, A, 400]
3	[T1, C, 300]
4	[commit, T1]
5	[start, T2]
6	[T2, B, 600]
7	[start, T3]
8	[T3, E, 900]
9	[T2, D, 50]
--- SYSTEM CRASH ---	

Recovery Manager Code (illustrative)

```
log = read_log('transaction.log')

committed, active = set(), set()
for rec in log:
    if rec.type == 'start': active.add(rec.txn)
    if rec.type == 'commit':
        committed.add(rec.txn); active.discard(rec.txn)

redo_list = committed   # NO-UNDO: active txns are simply ignored

for t in redo_list:     # forward chronological, from checkpoint
    for rec in writes_of(t):
        apply(rec.item, rec.new_value)

print('Discarded (never applied to disk):', active)
```

Sample Output (illustrative execution)

```
$ python3 deferred_update_recovery.py transaction.log
Committed transactions : [T1]
Active (uncommitted)   : [T2, T3]
UNDO -> none required (NO-UNDO/REDO)
REDO -> T1: A=400, C=300
```

```
Discarded (never applied to disk): {T2, T3}
Recovery complete.
```

Step 1: Classify Transactions

- T1: has [commit, T1] → COMMITTED.
- T2: wrote B and D, no commit record → UNCOMMITTED (active at crash).
- T3: wrote E, no commit record → UNCOMMITTED (active at crash).

Step 2: Apply NO-UNDO/REDO Rule

Under Deferred Update, a transaction's writes are held only in a memory buffer (and logged) — never applied to the database itself — until the transaction commits. This means the on-disk database can never contain an uncommitted transaction's changes, so UNDO is never required by construction:

- REDO list = { T1 } — re-apply A=400 and C=300 from the log, in case these committed writes had not yet been flushed to disk at crash time.
- UNDO list = { } (empty) — T2 and T3's writes were never applied to disk in the first place, so they require no reversal; the log records for T2 and T3 are simply discarded/ignored.

Step 3: Recovery Procedure

1. Scan the log forward from the last checkpoint.

2. For every transaction with a [commit, T] record, redo all of its writes using the new values in the log.

3. For every transaction with NO [commit, T] record, take no action — its changes were never externalized to the database.

4. Discard the incomplete log entries for T2 and T3; they may simply be restarted from scratch by the application if needed.

Explanation

The trade-off with Deferred Update is that every update has to sit in a memory buffer until commit time, which costs more RAM during execution — but in exchange, recovery gets much simpler, since nothing uncommitted ever touches disk and the recovery manager can skip the UNDO pass entirely and only ever REDO.

Justification

This approach is justified whenever recovery simplicity and speed are prioritized over run-time memory usage — REDO-only recovery is faster and less error-prone than the two-pass UNDO/REDO required by Immediate Update. The trade-off is that a transaction touching many items requires enough buffer space to hold all its updates until commit, which can be a constraint for very large transactions.

Question 9: Strict 2PL for a Banking System

Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.

Scenario

T1: Transfer(A → B, 500) — debits account A and credits account B. T2: ReadBalance(A) — reads the balance of account A while T1 is in progress.

Strict 2PL Rule

Under Strict 2PL, a transaction can acquire new locks throughout its execution (Growing Phase), but it must hold ALL of its locks — shared and exclusive — until it commits or aborts; no lock is released early. This differs from Basic 2PL, where locks may start being released as soon as the transaction enters its Shrinking Phase, before actually committing.

SQL Sessions under Strict 2PL (illustrative)

```
-- Session 1 (T1): fund transfer A -> B
mysql> START TRANSACTION;
mysql> UPDATE Accounts SET balance = balance - 500
    -> WHERE acc_id = 'A';
mysql> UPDATE Accounts SET balance = balance + 500
    -> WHERE acc_id = 'B';

-- Session 2 (T2): concurrent balance read
mysql> START TRANSACTION;
mysql> SELECT balance FROM Accounts WHERE acc_id = 'A'
    -> LOCK IN SHARE MODE;
-- (blocks: T1 holds X-lock on 'A' until commit)
```

Sample Output (illustrative execution)

```
-- Session 1
mysql> COMMIT;
Query OK, 0 rows affected

-- Session 2 (unblocks immediately after T1 commits)
+---------+
| balance |
+---------+
|     500 |
+---------+
1 row in set (waited 0.8 sec for T1 to commit)
```

Execution Timeline

Time	Txn	Action	Lock State
t1	T1	BEGIN	—
t2	T1	Lock-X(A); Read A; A = A - 500	X-lock(A) held by T1

Time	Txn	Action	Lock State
t3	T2	BEGIN	—
t4	T2	Request Lock-S(A)	BLOCKED — T1 holds X-lock(A)
t5	T1	Lock-X(B); Read B; B = B + 500	X-lock(A), X-lock(B) held by T1
t6	T1	COMMIT → release Lock-X(A), Lock-X(B)	locks on A, B released together
t7	T2	Lock-S(A) now granted; Read A	T2 proceeds only after T1's commit
t8	T2	COMMIT → release Lock-S(A)	—

Explanation

Because Strict 2PL forbids T1 from giving up any lock before it commits, T2's attempt to read A simply blocks until T1 is completely done — so by the time T2 finally sees a value, it's always the settled, post-transfer balance, never a half-applied intermediate one, which rules out dirty reads by construction.

Justification

Strict 2PL is specifically chosen (over Basic 2PL) for this banking scenario because it additionally guarantees recoverability and avoids cascading rollbacks: if T1 were to abort after T2 had already read its dirty intermediate value under Basic 2PL, T2 would need to be aborted too. By holding locks until commit, Strict 2PL ensures every value read by another transaction is already permanent, which is essential for financial correctness — a bank cannot allow a balance inquiry to reflect a transfer that later gets rolled back.

Question 10: Pseudocode for Heuristic Query Optimization

Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.

Pseudocode

```
FUNCTION optimizeQueryTree(tree):

    // Rule 1: Break conjunctive selections into individual nodes
    FOR each SELECT node sigma with condition (c1 AND c2 AND ... AND cn):
        REPLACE sigma with a cascade of n single-condition SELECT nodes

    // Rule 2: Push-Down Selection
    FOR each SELECT node sigma(condition) in tree:
        child = sigma.child
        WHILE child is a JOIN or CARTESIAN PRODUCT node:
            IF condition only references attributes from ONE side
               of child (left or right subtree):
                MOVE sigma below child, directly above that subtree
                child = new child (the subtree sigma moved into)
            ELSE:
                BREAK   // condition spans both sides; cannot push further

    // Rule 3: Replace Cartesian Product + Selection with Join
    FOR each node (SELECT(condition) over CARTESIAN(R, S)):
        IF condition is an equality on a common attribute of R and S:
            REPLACE the pair with a single JOIN(R, S, condition) node

    // Rule 4: Project Early
    FOR each node n in tree (bottom-up):
        neededAttrs = attributes required by n's parent(s) and n itself
        IF n is a leaf (base relation) OR a JOIN/SELECT node:
            INSERT a PROJECT(neededAttrs) node directly above n,
            keeping only columns needed further up the tree

    // Rule 5: Order joins by smallest estimated intermediate result
    joinList = extractAllJoinNodes(tree)
    FOR each join J in joinList:
        J.estimatedSize = estimateResultSize(J.left, J.right, J.condition)
    SORT joinList by estimatedSize ASCENDING
    REBUILD tree so joins execute in ascending order of estimatedSize
    // (smallest intermediate results are computed first, minimizing
    //  the input size to every subsequent, more expensive join)

    RETURN tree
```

Sample Output (illustrative execution trace)

```
$ python3 heuristic_optimizer.py sample_query.tree
Rule 1: split AND-condition into 3 individual SELECT nodes
Rule 2: pushed sigma(Location='Chennai') down to Department
```

```
Rule 2: pushed sigma(Salary>AVG) down to Employee
Rule 3: replaced Cartesian x + sigma(join-cond) with JOIN
Rule 4: inserted early PROJECT(Name, DeptName, DeptID)
Rule 5: join order sorted by estimated size -> unchanged
        (single join in this example)
Optimized tree written to optimized_query.tree
```

Explanation

Working directly on the relational-algebra tree, this algorithm applies five rewrite rules from the bottom up: it first breaks compound conditions apart so each piece can be relocated on its own, then pushes each SELECT down toward the leaf relations it actually needs, folds CARTESIAN-plus-SELECT combinations into proper JOIN nodes, adds PROJECT nodes early to narrow tuples as soon as possible, and finally reorders any multi-way joins so the one expected to yield the smallest intermediate result runs first.

Justification

Each rule is independently size-reducing and therefore safe to apply without cost estimation for Rules 1–4 (pushing selection/projection down can never increase the size of an intermediate result, only shrink it). Rule 5 is the one heuristic that still needs approximate size estimates (e.g., from table statistics), because join order genuinely affects total cost — computing the smallest joins first keeps every subsequent join's input relations as small as possible, which is why query optimizers apply Rules 1–4 unconditionally but treat join ordering (Rule 5) as a search problem guided by cost estimates.